A Working Grammar for Every Language

Carrying forty years of SIL's parsing work into the hands of every translation team, and beyond.

ቋንቋ لغة भाषा ဘာသာစကား

language

SIL Global · HermitCrab, PanGloss and Motif · October 2026

Contents

Introduction 3

Groundwork and the next step 5

If all five moved forward 10

Phase 1: Bible translation 15

Phase 2: Every language community 23

Risks, costs and next steps 30

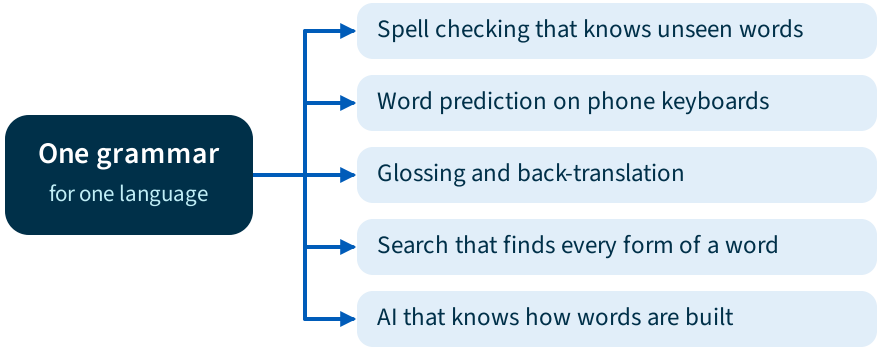
Glossary and sources 34

Introduction

A **grammar**, in the sense this paper uses, is a description of how the words of a language are built: its roots, its prefixes and suffixes, the order they come in, and the sound changes that happen when they meet. A **parser** runs that description backwards. Give it a word, and it tells you what the word is made of and what each piece means.

SIL has worked toward this for nearly forty years, from AMPLE and PC-KIMMO to Mike Maxwell's **HermitCrab**, which Damien Daspit built into FieldWorks and which John Maxwell and others continue to develop. It can model nearly any language on earth, and grammars are being built on it now, in FLExTrans projects and in parser workshops from Manila to Nairobi.

What those grammars need next is tooling that keeps pace as they grow: faster re-parsing, a clear view of what the grammar is doing, and ways to put a finished grammar to work beyond FieldWorks. This paper sets out a two-phase plan to provide it, building on HermitCrab rather than beside it.



What you'll find here:

- Forty years of groundwork, and the five things that decide how far a grammar reaches
- **Phase 1**, built for Bible translation inside SIL, which serves translation from day one
- **Phase 2**, built for every language community, which needs partners

Who this is for: SIL leadership deciding whether to fund Phase 1; language technology and Bible translation leaders who would deploy it; the FieldWorks and HermitCrab teams it builds on; and prospective partners for Phase 2.

Two ways to read this

In a hurry? Read the next page, then Chapter 3 (*Phase 1*) and Chapter 5 (*Risks, costs and next steps*). A shorter companion, *Motif in Brief*, covers the working demonstration that exists today, in nine pages.

THE ONE IDEA A grammar is software for a language. Once it is fast, easy to build and runs everywhere, every tool that handles text in that language gets better at once.

In one page

The pitch in one page

	Today	Phase 1 — Bible translation	Phase 2 — Every language community
Speed	Slows as a grammar grows: minutes to hours for a text	**50× faster (target)**: algorithm improvements shared with HermitCrab in SIL.Machine, plus what Rust adds: careful memory use and every processor core	**Near-instant, as fast as typing (goal, to be proven)**: grammars compiled into a much faster form
Easy to build	A trace per word; needs specialist support and years	Statistics, timing, progress and grammar-health diagnostics, with AI assistance, in FieldWorks	AI proposes grammar changes; statistical harvesting of the lexicon
Deployment	FieldWorks and FLExTrans, where people already publish	**FieldWorks** and **Paratext**: back-translations and spelling checks where translators work	Firefox, LibreOffice, Wikipedia, Keyman prediction, literacy apps
Languages using it	A committed few, many more in progress	**Hundreds**: a clear win for every Bible translation project	A **shared library** of grammars anyone can use
Completeness	Scripture first, where today's incentive lies	Scripture first	Grammar coverage checks and report cards for news, health, literacy and school

THE INVITATION Fund Phase 1 as a Bible translation tool, built together with the FieldWorks and HermitCrab teams. It is justified on that use alone. Treat Phase 2 as the long-term hope, and start the partner conversations now, because it needs them.



Chapter 1

Groundwork and the next step

Chapter 1

What a grammar can do

What a parser does

In much of the world a single word is a whole sentence. The Swahili word *hawatanipenda* means "they will not love me":

ha	wa	ta	ni	pend	a
not	they	will	me	love	(ending)

A verb can have thousands of forms; one study counts 2,253 possible forms of a single Finnish noun. A word list cannot hold them all, so spell checkers for these languages flag correct words and miss wrong ones. A parser knows the pieces and the rules, and works out any word from them.

HermitCrab

HermitCrab is SIL's rule-based morphological parser. It models what linguists describe: roots and affixes, templates of slots, and sound changes at the boundaries. It ships in **FieldWorks Language Explorer (FLEx)** alongside the older XAMPLE parser, and **FLExTrans** builds machine translation on FLEx grammars. In principle it can parse nearly every word in every language. The work of this paper is helping it do so in practice.

Why a grammar is worth having

A working grammar is not an end in itself. It is an engine that other tools switch on:

- **Spell checking** that understands words it has never seen
- **Word prediction** on phone keyboards, which matters most where typing is hardest
- **Glossing and back-translation**, so a consultant can read a draft they don't speak
- **Search** that finds every form of a word, not just the one typed
- **AI**, which is weakest exactly where text is scarce, and gains most from structured knowledge of a language



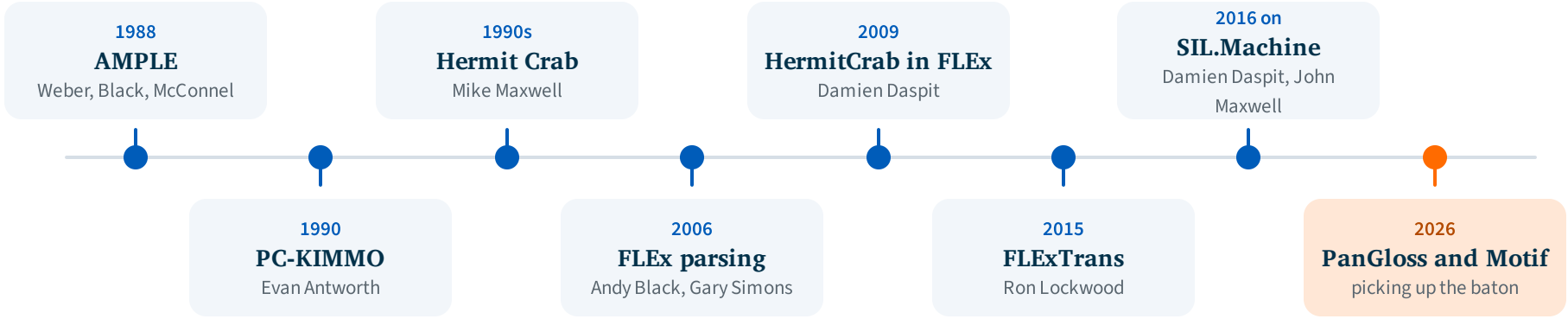
Ten of the 2,253 possible forms

THE OPPORTUNITY Every one of these tools depends on the same thing: a working grammar. For most of the world's languages that grammar is still being built, or not yet begun.

Chapter 1

Forty years of groundwork

Every grammar in FieldWorks today stands on work that began in the 1980s: David Weber, Andy Black and Stephen McConnel's AMPLE; Evan Antworth's PC-KIMMO; Mike Maxwell's Hermit Crab; the FLEx parsing Andy Black and Gary Simons described; the HermitCrab Damien Daspit built into FieldWorks; Ron Lockwood's FLExTrans; and HermitCrab's continuing life in SIL.Machine with Damien Daspit and John Maxwell.



Grammars being built now

- **FLExTrans** helps draft Scripture for language communities in the Philippines and Peru.
- **Parser workshops** train linguists to build grammars for their own languages: Manila in 2025, Nairobi in 2026.
- **Many more grammars** sit in FieldWorks projects at every stage. A few are mature; many are "getting there", built patiently over years.

Work continuing today

HermitCrab's maintainers are making it faster. Algorithm work in SIL.Machine is delivering 5–10× speedups, which PanGloss shares, and FieldWorks can now limit runaway parses and parse only the words that still need an analysis.

PICKING UP THE BATON This paper does not propose starting over. It proposes carrying this work the rest of the way: keeping every grammar already built, and giving the people building them tools that keep pace as their grammars grow.

Chapter 1

Where grammar-building stands

Five things decide how far a grammar can reach. Today's tools have carried grammars a long way on each; the next step on each is within reach.

1 · Speed

As a grammar grows, some combinations of rules become expensive to search. On one Amharic grammar a list of 7,000 words takes about half an hour, and one word in six hits a five-second limit. When re-parsing is slow it happens less often, and the grammar improves more slowly.

2 · Easy to build

Try a Word shows how the parser handled one word, but nothing shows, across a whole text, where the time goes or which rules never fire. Building a grammar still takes specialist support and patient years.

3 · Deployment

Grammars already do real work in FieldWorks and FLExTrans, and people publish with them. The aim is to broaden that: the other places people type, read and publish, from Paratext to word processors, browsers and keyboards, cannot yet use them.

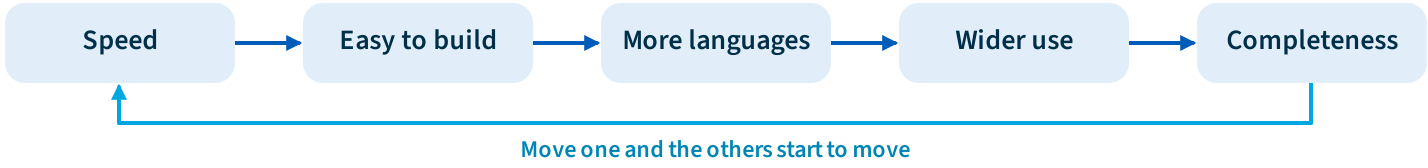
4 · Languages using it

A committed community of grammar builders, a handful of mature grammars and many more in progress. Ethnologue counts 7,159 living languages.

5 · Completeness

Nothing limits a grammar to one kind of text, but today's main incentive is Bible translation, so most grammars are built and tested on Scripture rather than the everyday language of news, health or school. That is a present reality, not SIL's hope.

WHY THIS MATTERS Faster parsing makes grammars easier to build; more languages encourage wider use; wider use gives a reason to make grammars complete. **Move one and the others start to move.**



Chapter 1

The foundation is sound; the next step is tooling

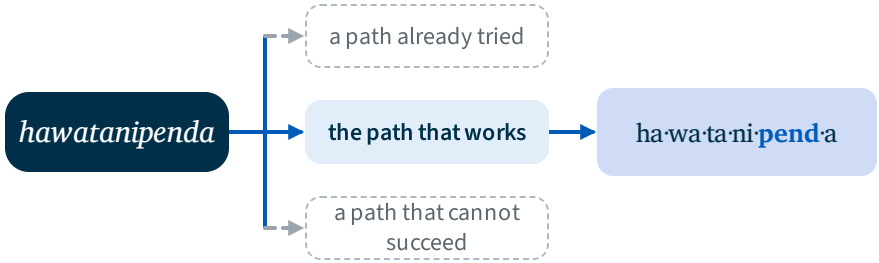
HermitCrab's model of language is sound, and decades of grammars show it. What grammar builders need next lies in the engineering and tooling around it, which is exactly the part that has become cheaper to build.

Slow in places because of how much it explores

To be sure of every answer, a parser considers many ways a word *could* have been built, and as a grammar grows much of that is wasted. HermitCrab's maintainers are already skipping it without changing an answer, with 5–10× speedups in SIL.Machine.

Hard to build because so much is hidden

Try a Word shows the parser's reasoning for one word. What is missing is the view across a whole text: which rules are expensive, which never fire, which let in forms the language does not have. Without it, improving a grammar relies on long experience reading traces.



In FieldWorks because of its runtime

HermitCrab is a C# library that runs inside FieldWorks, reading FieldWorks' own data. That is the right home for building a grammar, but Paratext, a browser or a keyboard app cannot load it.

Few languages, and Scripture first, because of the other three

A grammar is a long commitment, and today it pays back mainly in Bible translation. FieldWorks serves more than 1,300 language communities; far fewer have taken a grammar to the point where the parser does daily work.

WHAT CHANGED Three things. HermitCrab's maintainers have shown how much speed better algorithms can find. **PanGloss**, a faithful port of HermitCrab that runs anywhere, is under way. And AI tools have made this kind of software cheaper to build, and can help a linguist read a trace and propose a fix. **What remains is careful work, and it can now be done faster than before.**



Chapter 2

If all five moved forward

Chapter 2

If all five moved forward

1 · Near-instant

The goal, still to be proven: parsing as fast as typing, so a whole Bible re-parses in about the time it takes to save, and a keyboard can analyse each word as it is typed.

2 · Weeks, not years

Someone who knows their language well can make steady progress on a grammar in weeks, with statistics, progress bars and an AI assistant that explains what went wrong.

3 · Everywhere

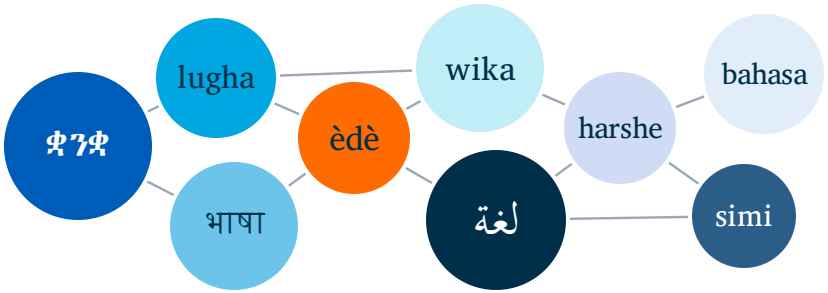
In Paratext, Firefox and LibreOffice, on language-analysis websites, in spelling prediction and correction, and in instant back-translations for people and for AI.

4 · Every language

Every language community that wants a working grammar can have one.

5 · Full content

Grammars that handle news, health information, literacy materials and primary school textbooks, not only Scripture.



SIL GLOBAL'S VISION

People flourishing in community using the languages they value most.

"Language" in nine languages

THE PICTURE A grammar becomes a gift a community gives to every tool that serves its language: built once, maintained by that community, and used wherever the language is written.

Chapter 2

A day in that world

A translator in Paratext

She finishes a draft of Mark 4. Before she saves, every word has been parsed. Three are flagged: two are typos, one is a new word the grammar has not seen. A literal back-translation appears beside her text, word by word, for the consultant who arrives next month and does not speak the language.

A primary school teacher

A teacher prepares a reading book on the class laptop. LibreOffice underlines words that are misspelled, and not the correct ones that happen to be long. His pupils practise spelling on a phone app built from the same grammar.

Watoto	**wanasoma**	**kitabu**
wa-toto	wa-na-som-a	ki-tabu
PL-child	they-PRES-read-FV	CL7-book

Literal children they-are-reading book

Free The children are reading a book.

Illustration in Swahili

A Wikipedia editor

An editor writes a health article in her own language. The browser suggests corrections, and the keyboard predicts the next word from forms it has never seen, because it builds them from their parts.

An AI system

A translation model working in a low-resource language asks the grammar what an unfamiliar word means, and gets its root and a gloss for each piece. The model's weakest languages now come with a precise, human-checked description.

WITHIN REACH Each scenario uses tools that exist today: Paratext, LibreOffice, Firefox, Keyman, Wikipedia. What is missing is a fast, portable grammar for them to call.

Chapter 2

Why now

The foundation exists

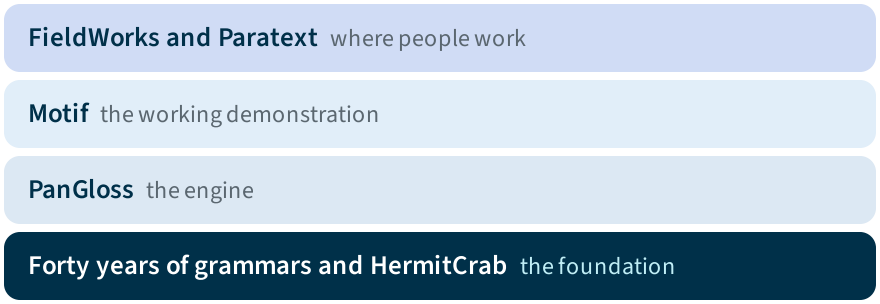
HermitCrab, and the grammars built with it over years, give a proven model and real data to work from. Its maintainers are making it faster.

The engine exists

PanGloss ports HermitCrab to Rust, sharing algorithm improvements and checking behavior against grammar fixtures. Rust adds careful memory use, parallel word batches and a portable engine; broader host deployment remains ahead.

A working demonstration exists

Motif shows which words parse, where the time goes and what to inspect. Invited testers have a Windows installer and macOS and Linux previews. It remains a demonstration where ideas meet real grammars before moving into FieldWorks.



The data exists

Translation work is in progress in 4,457 languages, and many teams keep lexicons and interlinear texts in FieldWorks: much of a grammar's raw material.

Building software has become cheaper

PanGloss and Motif were begun in the summer of 2026 by a small team working with AI coding agents. The constraint is no longer engineering capacity but deciding what to build, with the people who know these grammars best.

AI makes grammar-building more accessible

An assistant can explain a parser trace and propose a fix. Current Motif development adds an opt-in connection: draft supported changes, try them on a copy, inspect the differences and revise. A person always Applies.

PHASE 1 SERVES TRANSLATION FROM DAY ONE Bible translation already needs faster checking, back-translations and consistent spelling. Everything Phase 2 needs is built on the same foundation.

Chapter 2

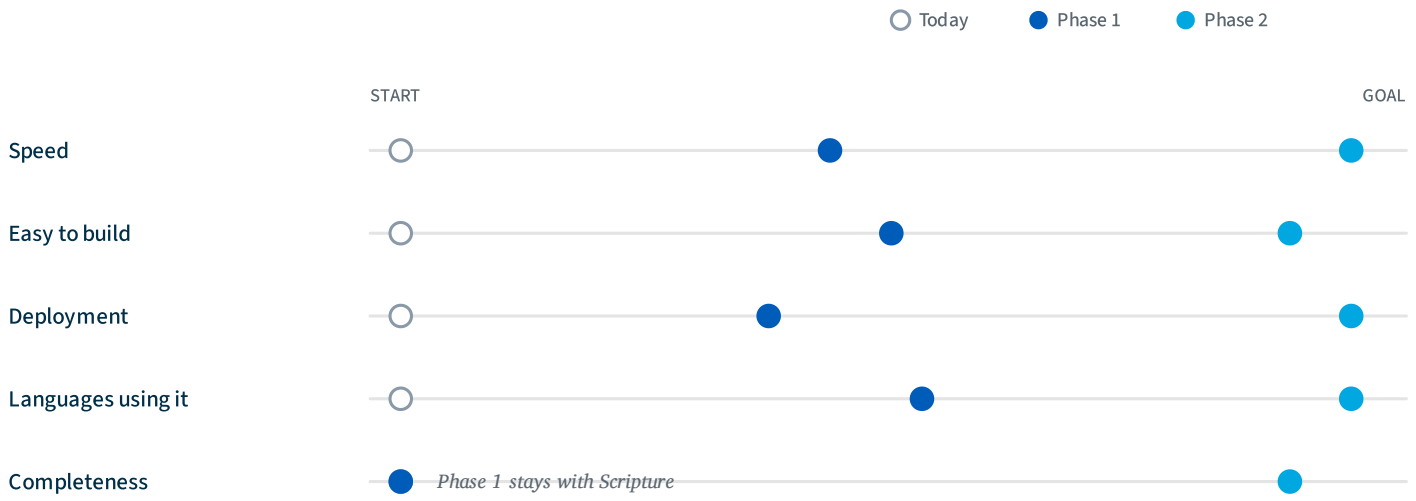
Two phases

Phase 1 · Usable for Bible translation within SIL

Built with Bible translation in mind, so its value is easy to judge, and delivered through SIL's own software: FieldWorks, where grammars are built, and Paratext, where translators work. PanGloss makes grammars faster; the tools demonstrated in Motif make them easier to build. **Justified by what it gives translation.**

Phase 2 · Usable for the flourishing of all language communities

Grammars leave the translation office: into browsers, word processors, keyboards, encyclopedias, literacy apps and a shared library of grammars. Grammar coverage expands from Scripture to everyday language. Its work over large bodies of parallel text may live in a companion app rather than in FieldWorks. **Needs external partners, if only for broader acceptance.**



HOW THE PHASES CONNECT Phase 2 needs no new theory, only more reach. Every Phase 1 grammar is a Phase 2 grammar waiting to be published, and every Phase 1 tool is the foundation Phase 2 extends.



Chapter 3

Phase 1: Bible translation

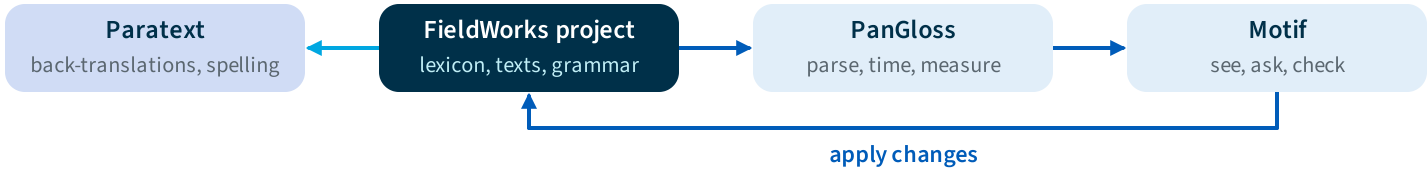
Chapter 3

Phase 1 at a glance

Usable for Bible translation within SIL.

	Today	Phase 1	How
Speed	Slows as a grammar grows	**50× faster (target)**	Shared HermitCrab and SIL.Machine algorithms, plus Rust memory use and parallel word batches
Easy to build	A trace per word; specialist support and years	**Stats, timing, progress and health diagnostics, with AI help**	PanGloss measures; Motif demonstrates the views on real grammars, and they move into FieldWorks
Deployment	FieldWorks and FLExTrans	**FieldWorks and Paratext**: faster parsing where grammars are built; back-translations and spelling detection where translators work	PanGloss runs as a single native program or library, with no FieldWorks dependency
Languages	A committed few, many in progress	**Hundreds (goal)**	A clear win for any translation team that already has a FieldWorks lexicon
Completeness	Scripture first, where today's incentive lies	Scripture	By design: Phase 1 stays focused on the use that justifies it; nothing stops a team going wider

PHASE 2 WAITS Broader deployment, a public grammar library and near-instant compiled parsing. Phase 1 stays focused on Bible translation, so it is quick to judge.

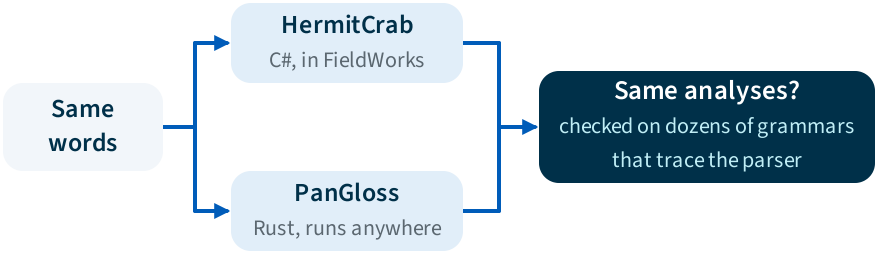


PanGloss: the engine

PanGloss carries HermitCrab into a form that runs anywhere. *Words in, morphemes out.*

Faithful where it matters

PanGloss ports HermitCrab's algorithms and checks their behavior against its conformance fixtures. It opens FieldWorks projects directly, with no export step. Documented differences include fixes proposed upstream; XAMPLE comparisons use separate FieldWorks witnesses.



Where the two differ, the difference is written up and tracked.

Faster, never by guessing

The rule is strict: **a speedup may cost memory or start-up time, never a correct answer.** Measurements name their grammar and word set; a result on one is not a promise for every language.

- **Shared with HermitCrab.** Better algorithms are being built in HermitCrab and SIL.Machine. PanGloss follows them, and a speedup found on either side can move to the other.
- **Parallel work.** Batches now spread words across several threads; Motif uses at most half the machine. One 300-word Mbugwe batch went from 1,608 s to 424 s with identical analyses. A difficult single word still sets the floor.

Deployable anywhere

Rust builds a single native program for Windows, macOS and Linux, with no runtime to install. The engine is built to be used as:

Surface	For
`pangloss` command line	Scripts, CI, AI agents, Motif
C library interface	FieldWorks, Paratext, other native hosts
WebAssembly	Browsers and web pages
Language Pack (`.pgpack`)	One data-only file holding a compiled grammar: no code, so it is safe to share

Native command-line releases now exist for Windows, macOS and Linux. The other surfaces exist in source; host integration and broader deployment remain ahead.

Near-instant parsing waits for Phase 2

Compiling a grammar into a much faster form could make parsing near-instant. The research is promising but unproven, so Phase 1 does not depend on it. It is the centrepiece of Phase 2.

WHY A PORT The aim is to keep the grammars people already built. Behavioral checks and explicit limits let an author evaluate the engine on their own data.

Stats, health and timing

A grammar author cannot fix what they cannot see. Try a Word shows how one word was parsed; PanGloss adds three measures across a whole word list.

Timing

Every word is timed. The slowest words are listed, and a runaway word is stopped at a fixed search limit and reported, instead of hanging the batch.

Statistics

Run over a representative word list, the statistics answer the questions an author actually asks:

- Where did the time go, and in which parts of the grammar?
- Which rules never produced anything?
- Which rules build forms that no root can finish?

Grammar health

`grammar-health` reports structural problems and rules behind stored analyses that no longer parse. It keeps reporting when a comparison cannot finish, and says what it did not measure.

Grammar health · illustrative

Error	**Stem with no category**	Can attach almost anywhere, and slows every word
Warning	**Affix with no slot**	Tried where it can never fit
Info	**Two phonemes that only look identical**	One may be skipped, or the two confused

- **A partial morpheme.** A stem with no category, or an affix with no slot, can attach almost anywhere. The parser then explores far more paths, and **one such entry can slow down every word**, not just the words that use it.
- **Two phonemes that look identical** on screen but are stored differently, so one is silently skipped or the two are confused.

THE PRINCIPLE A diagnostic reports evidence; it does not edit the grammar. An unfinished search never proves that a reading is absent.

WHY IT MATTERS Statistics make improvement measurable. Motif's new Parsimony checks add evidence about duplication and broad conditions, with the decision left to the linguist.

Motif: a working demonstration

Motif is a demonstration, not a finished product. It shows on real grammars what these tools could be, so that the ones that prove themselves can move into FieldWorks. It is a desktop application and a command-line tool with the same abilities, working on the FieldWorks project the translation team already uses.

What the demonstration shows today

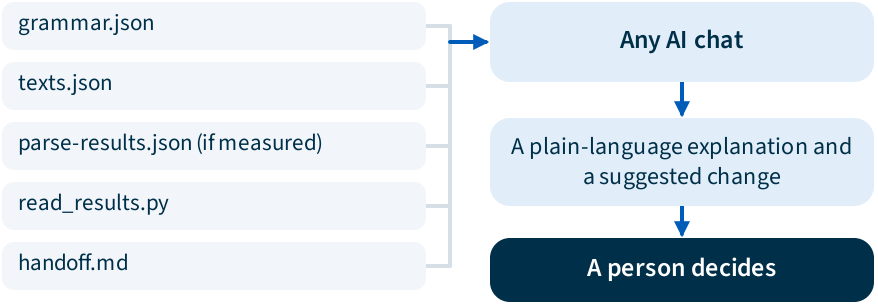
- **Overview**: how much of the chosen texts the grammar parses, how accurately, and how fast
- **Texts**: every word in context, parsed or not
- **Try a Word**: the parser's reasoning for one word, step by step, including where it gave up
- **Timing**: the slowest words and the rules that cost the most
- **Warnings**: the grammar-health report, ready to act on
- **Review changes**: check marked changes, then apply them to the FieldWorks project
- **AI Handoff**: the whole picture, packaged for an AI assistant

Motif runs PanGloss as a separate process, so a slow word never freezes the window.

BUILT FOR REVIEW A change is checked before it lands, not cleaned up after. That is what makes it safe to let an AI suggest changes to a grammar a translation team depends on.

Two ways to work with an assistant

AI Handoff writes chosen grammar, texts, available results and diagnostics, a helper script and a guide for a person to share. Current development also connects Claude and Codex through opt-in tools: an assistant can draft, try and revise supported changes, but cannot Apply.



You choose what to share; results are optional.

Changes, checked before they land

An author marks analysis changes in Texts: approve or disapprove an analysis. Motif tries them on a copy (**Dry Run**), re-measures, then Applies in one step with a receipt, while FieldWorks is closed. Supported grammar changes now come as **Proposals**, with semantic intent and a **Trial** measuring their effects. The complete before-and-after window review is still being finished.

From demonstration to FieldWorks

Motif is not meant to become one more program for a translation team to install. What proves itself there is meant to move into FieldWorks, where grammars are already built.

Phase 1: into FieldWorks

Motif a working demonstration

- PanGloss, callable from FieldWorks
- Assessment and the Overview
- Try a Word, Timing and Warnings
- Checking and applying changes
- The AI Handoff

what works moves home →

FieldWorks
where grammars are built

Phase 2 parallel texts and word harvesting: in FieldWorks, or a companion app? Still open.

How the move stays small

Motif is built on .NET 10 and Avalonia, the same move FieldWorks' own interface work is making, and keeps its views and command core separable, so that a later merge is a move, not a rewrite. Until then, the two are designed to meet at one point: at a save, FieldWorks releases the project, asks Motif to apply the pending changes, and reloads.

Phase 2: perhaps its own app

Phase 2 works with parallel texts and statistical word harvesting: tens to hundreds of megabytes of outside text that does not belong in a FieldWorks project. Whether that work lives in FieldWorks or in a companion app is still open, and is a decision to make with the FieldWorks team.

Built with the people who know these grammars

HermitCrab's maintainers and FieldWorks' developers know these grammars, and the people who build them, better than anyone. Speedups, diagnostics and interface work should flow both ways.

A DEMONSTRATION, NOT A NEW PRODUCT Motif lets new ideas meet real grammars quickly, without putting a stable, widely used FieldWorks at risk. What works goes home to FieldWorks.

The grammar-improvement loop

PanGloss measures, Motif shows, an AI assistant drafts, and a person decides. Analysis changes already follow this loop. In current development, connected assistants can draft, Dry Run, Trial and revise supported grammar changes before Finalize. Phase 1 must prove that workflow on real projects and finish its window review.



What the author needs

A good understanding of their language, and the judgement to say whether a proposed change is right. *Not* years of training in computational linguistics.

What makes it fast

Parallel Assessments; diagnostics that point at the problem; an assistant that reads traces so the author does not have to; changes that are safe to try.

What keeps it honest

The grammar is tuned, never the parser. Evidence names the exact change and project state it measured; the assistant cannot Apply, and a person decides.

Putting grammars to work in Paratext

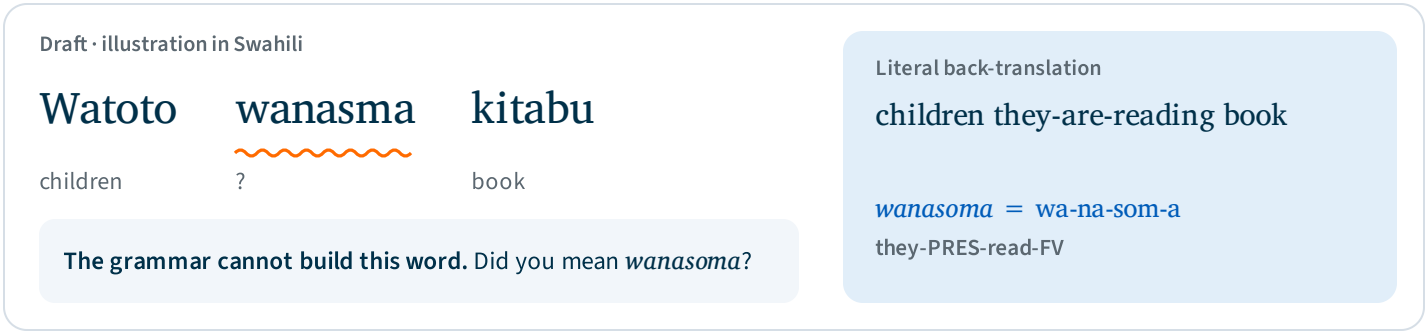
Paratext is where Bible translation happens: more than 15,000 users in nearly 550 organisations. It already has a word list, a spelling-status workflow, a prefix-and-suffix morphology check and a word-for-word interlinearizer. Phase 1 puts a real grammar behind them, as two features translators feel on the first day.

Automatic back-translations

Each verse gets a literal, word-by-word gloss built from the grammar's analysis: root meaning plus what each affix contributes. Consultants read a draft in a language they don't speak; teams spot where a word does not say what they meant. Later work turns morpheme glosses into natural phrases.

Spelling detection

A word the grammar cannot build is flagged. Unlike a word list, the grammar accepts the thousands of correct forms no list contains, and catches the misspellings a list would miss. Inconsistent spellings of the same word across a book are grouped for the team to resolve.



Why hundreds of languages, not a handful

Any translation team with a FieldWorks lexicon and some interlinear text already has most of a grammar's raw material, and many already have a grammar under way. With better diagnostics and an AI assistant, turning that into a working grammar becomes a project of weeks. The reward, back-translations and spelling checks, is something every team wants. **For Bible translation, this is a clear win.**

Why it serves translation from day one

Consultant checking, back-translation and spelling consistency already cost translation projects real time and money. Phase 1 reduces all three, using software SIL already stewards, on content SIL already works with.



Chapter 4

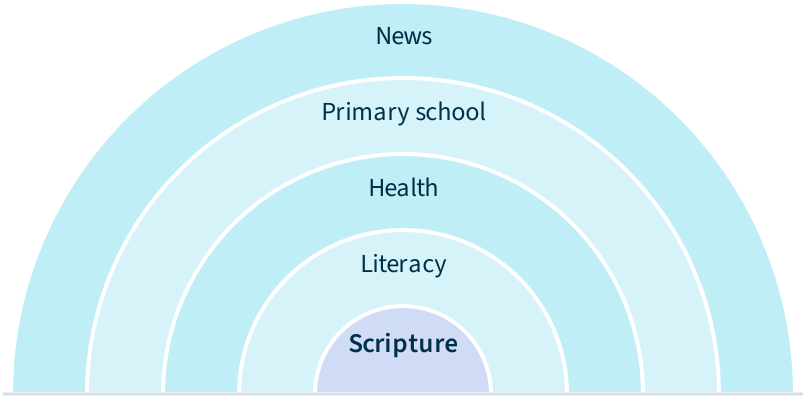
Phase 2: Every language community

Chapter 4

Phase 2 at a glance

Usable for the flourishing of all language communities.

	Phase 1	Phase 2	How
Speed	50× faster (target)	**Near-instant, as fast as typing (goal, to be proven)**	PanGloss 2.0 compiles a grammar into a faster form that suggests answers almost at once; the full engine checks each one
Easy to build	Diagnostics and AI help	**AI generates changes; the lexicon is harvested statistically**	Motif 2.0, perhaps as a companion app, learns from parallel texts across many domains and integrates statistical tools
Deployment	FieldWorks and Paratext	**Wikipedia, Firefox, LibreOffice, Keyman prediction, literacy apps**	Language Packs that any host can load, published through a shared library
Languages	Hundreds	**Every community that wants one**	A shared library where communities publish grammars for spell checkers, predictors and more
Completeness	Scripture	**News, health, literacy, primary education**	Grammar coverage checks and report cards that show where a grammar is thin



Coverage grows outward

Phase 1 · Scripture

Phase 2 · literacy, health, school, news

PanGloss 2.0: as fast as typing?

The goal, still to be proven

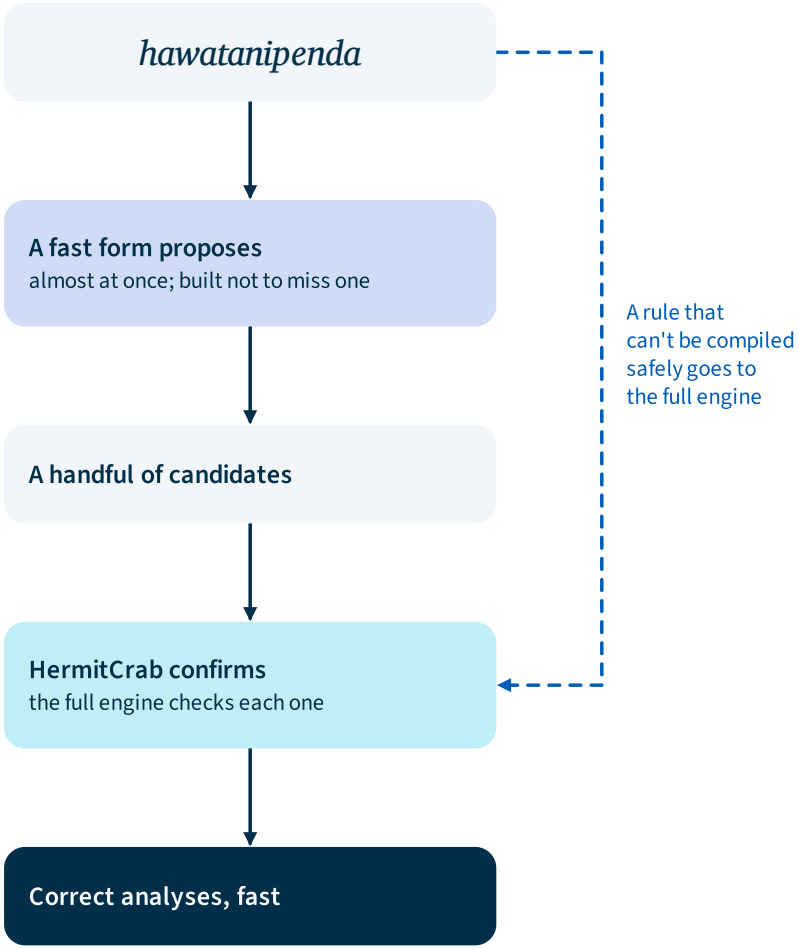
Phase 2 aims for parsing that feels instant: fast enough to check each word as it is typed, on a phone as well as a desktop. The fastest analysers in the world get there by compiling a grammar ahead of time into a compact, quick form. Whether PanGloss can do the same for FLEx grammars, across everything they describe, is what Phase 2 has to show.

Fast suggestions, checked answers

A compiled grammar can be subtly wrong, because some rules are hard to capture in one. PanGloss 2.0 is designed to avoid that risk with **propose-and-confirm**: the fast form *proposes* likely analyses almost at once, and the full HermitCrab engine *confirms* each one.

The design requires the fast form to propose every valid analysis: confirmation cannot recover one it missed. Today unsupported shapes can refuse compilation. Phase 1 uses the full engine; broad, safe acceleration is still to be proven.

EARLY, NOT PROVEN Early experiments are encouraging, with matching answers on the words compared. The approach has not yet been tested across real translation grammars, and some parts of a grammar still need the full engine.



PanGloss 2.0: spelling and prediction

From detection to correction

Detection says a word is wrong. Correction says what was meant. It is designed to layer five signals, each meant to be cheap enough to run as someone types:

1. **Likely typos**: common slips matched quickly to real words
2. **Keyboard distance**: nearby keys on the actual layout make a slip more likely
3. **Sound-alikes**: spelling-by-ear errors, matched through the language's own sound rules
4. **Diacritics**: missing or misplaced accents and tone marks, restored only to words the grammar can build
5. **Context**: which words usually appear together in real text

Restoring diacritics

French checkers such as LanguageTool and Antidote already flag missing accents, and a Keyman keyboard can suggest *naïve* for *naiv*. In many minority languages the marks carry meaning: in Yorùbá and Igbo, tone marks tell apart words otherwise spelled alike, yet people often type without them. A grammar adds what a word list cannot: it proposes only the marked forms it can build, and a tone rule it encodes rules out the rest. Where two marked words are both valid, context decides. Tidying how text is stored cannot do this: it never restores a missing mark.

Built from lots of text

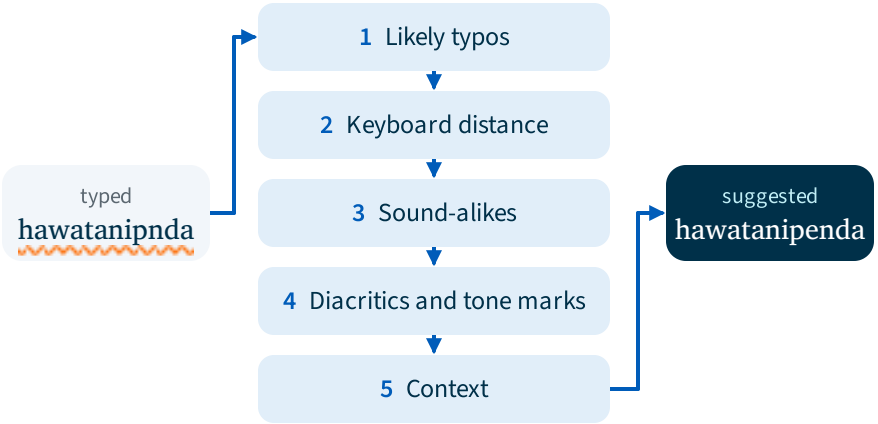
PanGloss 2.0 learns from large text collections which words occur, how they are analysed and which appear together, so correction and prediction reflect how the language is really written.

Prediction

A keyboard that predicts words needs to know which words exist. Keyman's predictive text today is built from word lists with frequencies. A grammar can generate those lists, including the valid forms a collected list would miss, and a later integration can ask the grammar directly.

Hosted

The same engine in WebAssembly powers language-analysis websites: paste text, see every word analysed, no installation.



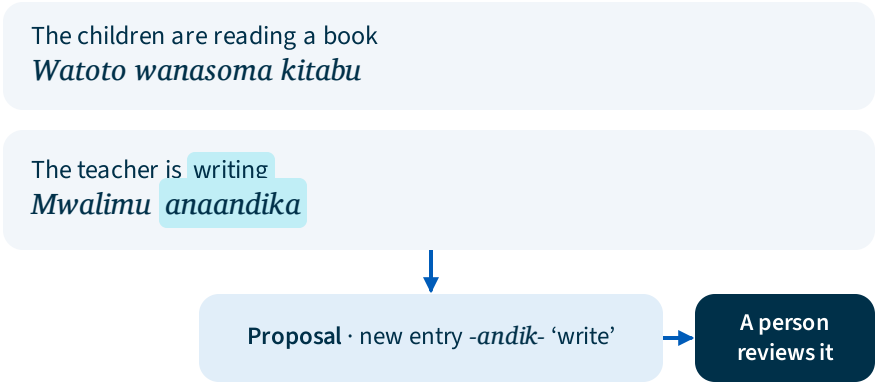
WHY THIS MATTERS FOR SMALL LANGUAGES Where little text has been published, there is not enough data to learn spelling statistically. A grammar supplies what the data cannot.

Motif 2.0 and a shared library

Phase 2's work over large bodies of text may live in its own app rather than in FieldWorks. *Motif 2.0* names that work wherever it lands.

AI that proposes the grammar

In Phase 1, the AI explains and drafts one change at a time. In Motif 2.0 it works from **parallel text**: translations of the same content in the language and in a language of wider communication, across many domains. From them it proposes new lexical entries and rules as Proposals, which people review exactly as before.



Statistical harvesting of the lexicon

Words the grammar cannot yet parse are the work queue. Statistical tools cluster them, suggest likely roots and affixes, and rank them by how often they occur, so authors spend their time where it matters most.

Publish to the library

When a grammar is good enough, its community can publish it as a Language Pack, with its report card attached.

A shared library of grammars

A public home for grammars, looked after by SIL or a partner such as the Wikimedia Foundation, where language communities publish grammars and any tool can use them:

- **Spell checkers** in Firefox, LibreOffice and other editors
- **Predictors** in Keyman keyboards and on phones
- **Glossers** for websites, publishers and AI systems

A Language Pack contains data only, never executable code, so a host can load one from a stranger without running a stranger's program.

OWNERSHIP A language's grammar belongs to its community. The library makes sharing easy; it must also make it the community's choice. Licensing and consent are part of the design, not an afterthought.

Where the grammars land

Firefox and LibreOffice

Spell checking that understands how words are built, through the same engine in WebAssembly and as a native library.

Keyman prediction

SIL's keyboard platform, which supports typing in more than 2,500 languages, suggesting whole words built from their parts.

Wikipedia

Editing help across 364 language editions; in a small language, each article is a real share of everything written in it.

Literacy and spelling app

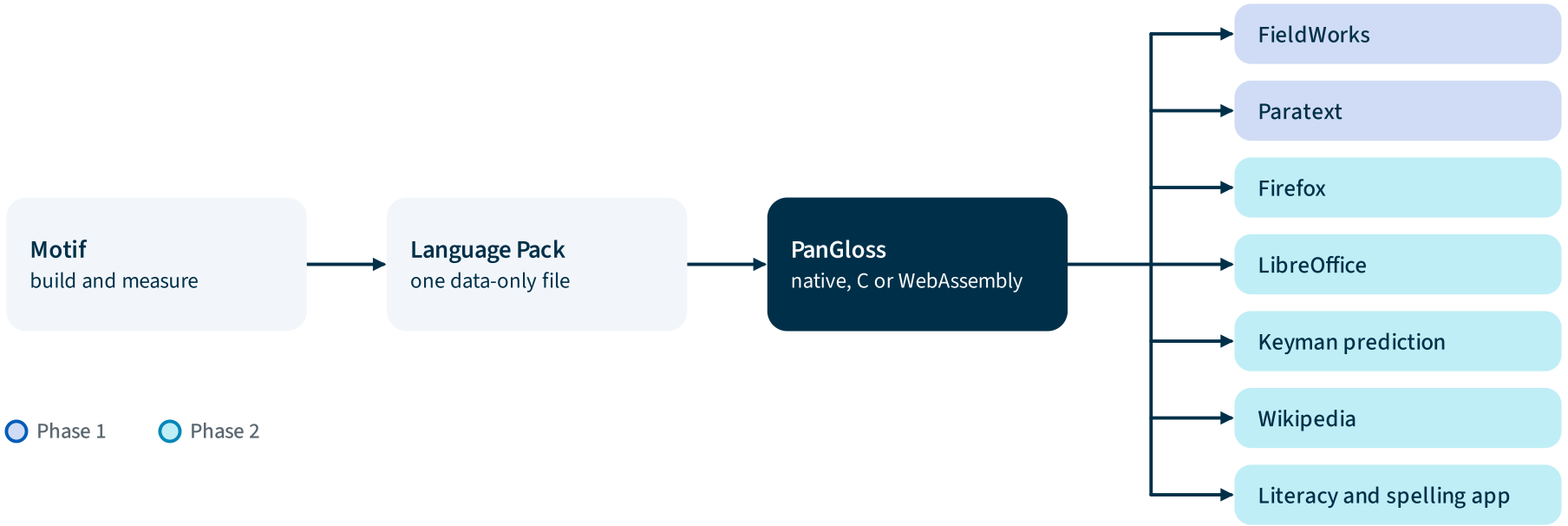
Helps learners practise their language's standard spelling, tone marks included, and helps publishers apply it.

Language-analysis websites

Paste a text and see every word analysed, for teachers, researchers and curious speakers.

AI back-translation

Instant, structured back-translations for people and for AI systems working in languages they barely know.



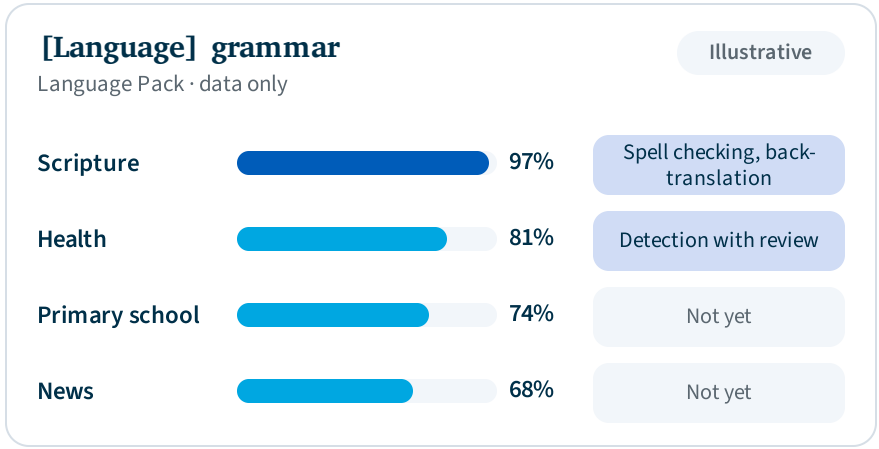
Report cards for completeness

A grammar built on Scripture knows Scripture's words. A health leaflet or a maths textbook will use many it has never seen. Phase 2 measures this directly, so a community knows what its grammar is ready for.

Three measures of reach

- **Text Coverage** (in the Motif demonstration today): what share of the words in the chosen texts the grammar parses
- **Grammar coverage**: the same measure over a named collection of texts, so a health leaflet and a gospel can be scored apart
- **Feature coverage** (designed): which combinations of affixes and slots the texts actually use, and which the grammar allows that no text shows. Each leftover is either a rule that is too broad or a word nobody has collected yet, and a person decides which

A report card



Report cards travel with the Language Pack, so a host can decide whether a grammar is good enough for its use, and a community can see where to work next.

BROADER ADOPTION Report cards turn "is this grammar any good?" into a question with a published answer, which is what partners outside SIL will need before they ship it.

Chapter 5

Risks, costs and next steps

Chapter 5

What could go wrong

Grammar-building is still skilled work

AI and diagnostics lower the bar; they do not remove it. **Mitigation:** Phase 1 starts with teams that already have FieldWorks data, a grammar under way and linguistic support, and measures how long a grammar really takes.

Speed targets are targets

50× and near-instant parsing are goals, not measurements across real grammars, and some parts of a grammar may never compile safely. **Mitigation:** publish a benchmark on real translation grammars at each milestone; whatever resists compiling takes the full engine, costing speed, never answers.

Working apart

Building beside HermitCrab instead of with it would split effort and lose decades of knowledge. **Mitigation:** plan with the SIL.Machine and FieldWorks maintainers from the start; conformance keeps PanGloss answering to HermitCrab, and improvements flow both ways.

Adoption outside SIL

Browser makers and encyclopedias will not ship tools they cannot evaluate. **Mitigation:** report cards, data-only Language Packs, open formats, and early partner conversations.

Prior art

Giellatekno and Divvun have built fast spelling and language tools for minority languages since 2001, now across more than 100 languages. **Mitigation:** PanGloss is complementary: it starts from grammars FLEx users already have, and checks every fast answer against the full engine. We learn from their best techniques.

Community ownership

Publishing a grammar is publishing a community's knowledge. **Mitigation:** consent and licensing built into the shared library from the start.

Chapter 5

What it costs, and who pays

Phase 1: SIL, justified by translation

Phase 1 builds on work already under way. PanGloss runs today, and Motif runs as a working demonstration; what remains is finishing PanGloss, moving what Motif has demonstrated into FieldWorks, integrating with Paratext, and supporting the first translation teams as they build grammars.

With modern tools the engineering is a small team working alongside the FieldWorks and HermitCrab maintainers, not a department. The larger cost is linguistic: supporting teams as they take their grammars further.

Its return is measured in consultant time, faster checking and better spelling in translation projects, which SIL already pays for today.

Phase 1 · SIL, with the FieldWorks and HermitCrab teams
justified by what it gives Bible translation

Phase 2 · SIL with partners
Wikimedia Foundation · Mozilla · The Document Foundation · Literacy and education organisations · Funders of language technology

Phase 2: SIL with partners

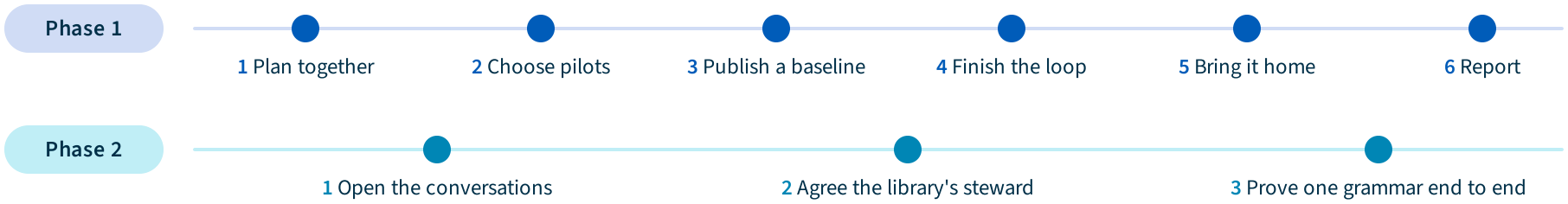
Phase 2 reaches beyond SIL's own software, so it needs others:

- **Wikimedia Foundation** for Wikipedia, and possibly as a neutral home for the shared library
- **Mozilla** and **The Document Foundation** for Firefox and LibreOffice
- **Literacy and education organisations** for the spelling app and school content
- **Funders of language technology and development**, since the benefits are in literacy, health and education, not only in translation

IF ONLY FOR ACCEPTANCE Even where SIL could build a piece alone, a partner's name is what makes a browser, a ministry of education or a publisher trust it.

Chapter 5

Next steps



For Phase 1

1. **Plan together.** Agree a shared plan with the FieldWorks and HermitCrab maintainers, so speedups and tools flow both ways.
2. **Choose pilot projects.** Five to ten Bible translation teams with a FieldWorks lexicon, interlinear texts, a grammar under way and a willing consultant.
3. **Publish a baseline.** Measure today's parse speed and Text Coverage on their data, so every later claim has a before.
4. **Prove the loop.** Finish the before-and-after window review and measure the assistant-to-Proposal workflow with pilot teams.
5. **Bring it home.** Move what Motif has demonstrated into FieldWorks, and ship back-translations and spelling detection in Paratext to the pilot teams.
6. **Report.** Time to a working grammar, speedup achieved, and what translators and consultants say.

For Phase 2

1. **Open the conversations now.** Wikimedia, Mozilla, The Document Foundation, Keyman, and one literacy partner.
2. **Agree the shared library's steward** and its licensing and consent model.
3. **Prove one grammar end to end**: from FieldWorks to a spell checker in LibreOffice and a Keyman predictor, with a report card.

THE DECISION IN FRONT OF US Phase 1 is a Bible translation tool that serves translation from day one and builds on decades of SIL's work. Say yes to it, and the foundation for a working grammar for every language gets built along the way.

Appendix

Glossary

Affix A piece added to a root: a prefix, suffix, infix or circumfix.

AMPLE SIL's morphological parser from 1988, and the basis of XAMPLE.

Assessment A stored PanGloss measurement of one kind; unfinished searches are recorded as incomplete.

Dry Run The actual effects of a Proposal, read back from a throwaway copy of the project before Apply.

FieldWorks (FLEx) SIL's desktop software for lexicons, texts and grammars.

FLExTrans A machine-translation system built on FieldWorks data.

Grammar A description of how the words of a language are built.

Handoff Motif's package of grammar, words, texts and traces for an AI assistant.

HermitCrab SIL's rule-based morphological parser, designed by Mike Maxwell and built into FieldWorks by Damien Daspit; now part of SIL.Machine.

Keyman SIL's keyboard platform, with predictive text.

Language Pack One data-only file holding a compiled grammar, loadable by any PanGloss host.

Morpheme The smallest piece of a word that carries meaning.

Motif A working demonstration of tools for building, measuring and changing grammars; what proves itself moves into FieldWorks.

PanGloss A port of HermitCrab to Rust: the same algorithms, portable and measurable.

Paratext SIL and the United Bible Societies' software for Bible translation.

Parser Software that works out what a word is made of.

Proposal Named intent to change a project, authored as a Draft and applied as one unit.

Trial A Proposal tried on a copy: its Dry Run plus one or more Assessments.

XAMPLE FieldWorks' AMPLE-based morphological parser.

Appendix

Sources and further reading

History

- Weber, Black and McConnel, *AMPLE: A Tool for Exploring Morphology*, SIL, 1988
- Antworth, *PC-KIMMO: a two-level processor for morphological analysis*, SIL, 1990
- Maxwell, "Two Theories of Morphology, One Implementation", SIL Electronic Working Papers 1998-001
- Black and Simons, "The SIL FieldWorks Language Explorer Approach to Morphological Parsing", 2006
- FieldWorks 6.0 announcement, FLEx list, May 2009
- FLExTrans · HermitCrab in SIL.Machine

The projects

- PanGloss: README, the grammar diagnostics guide and the spell-checking plan
- Motif: current architecture, beta release notes, shared Help, and the agent edit-loop and Proposal-review decisions

Measurements quoted

- Half an hour for 7,000 Amharic words, one in six stopped at five seconds: a Motif cross-repository measurement, August 2026
- Mbugwe parallel batch, 1,608 s to 424 s for 300 words: Motif beta002 release notes, October 2026; one 20-processor Windows machine
- 5–10× from algorithm work: HermitCrab development in SIL.Machine, 2026

Background figures

- 7,159 living languages: Ethnologue, 28th edition (2025)
- 4,457 languages with translation in progress: Wycliffe Global Alliance (August 2025)
- 15,000+ Paratext users, ~550 organisations: paratext.org
- 2,500+ languages in Keyman: keyman.com
- 364 Wikipedia editions: Wikimedia Meta (September 2026)
- 2,253 forms of a Finnish noun: COLING 2018 paper

Background

- FieldWorks · Paratext · Keyman
- Divvun and Giellatekno: finite-state language technology for minority languages
- Chapter maps: Natural Earth (public domain)